

Introduction to Computer Architecture

Sheet #7

Exercises:

1. Assume you have a machine that uses 32-bit integers and you are storing the hex value 1234 at address 0.
 - a. Show how this is stored on a big endian machine.
 - b. Show how this is stored on a little endian machine.
 - c. If you wanted to increase the hex value to 123456, which byte assignment would be more efficient, big or little endian? Explain your answer.

Ans.

a and b.

Address →	00	01	10	11
Big Endian	00	00	12	34
Little Endian	34	12	00	00

- c. Little endian is more efficient because the additional information simply needs to be appended. With big endian, the "12" and "34" would need to shift to maintain the correct byte ordering.

2. Show how the following values would be stored by machines with 32-bit words, using little endian and then big endian format. Assume each value starts at address 10_{16} . Draw a diagram of memory for each, placing the appropriate values in the correct (and labeled) memory locations.
 - a. $456789A1_{16}$
 - b. $0000058A_{16}$
 - c. 14148888_{16}

Ans.

a.

Address →	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	45	67	89	A1
Little Endian	A1	89	67	45

b.

Address →	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	00	00	05	8A
Little Endian	8A	05	00	00

c.

Address →	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	14	14	88	88
Little Endian	88	88	14	14

3. The first two bytes of a 2M x 16 main memory have the following hex values:
- Byte 0 is FE
 - Byte 1 is 01

If these bytes hold a 16-bit two's complement integer, what is its actual decimal value if:

- a. Memory is big endian?
- b. Memory is little endian?

Ans.

$$\text{a. } \text{FE01}_{16} = 1111\ 1110\ 0000\ 0001_2 = -511_{10}$$

$$\text{b. } \text{01FE}_{16} = 0000\ 0001\ 1111\ 1110_2 = 510_{10}$$

4. Convert the following expressions from infix to reverse Polish (postfix) notation.
- a. $X * Y + W * Z + V * U$
 - b. $W * X + W * (U * V + Z)$
 - c. $(W * (X + Y * (U * V))) / (U * (X + Y))$

Ans.

$$\text{a. } XY * WZ * VU * ++$$

$$\text{b. } WX * WUV * Z + * +$$

$$\text{c. } WXYUV * * + * UX Y + * /$$

5. Convert the following expressions from reverse Polish notation to infix notation.
- a. $WXYZ - + *$
 - b. $UVWXYZ + * + * +$
 - c. $XYZ + VW - * Z + +$

Ans.

$$\text{a. } W * (X + Y - Z)$$

$$\text{b. } U + (V * (W + (X * (Y + Z))))$$

$$\text{c. } X + ((Y + Z) * (V - W) + Z)$$

6. a. Write the following expression in postfix (Reverse Polish) notation. Remember the rules of precedence for arithmetic operators!

$$X = \frac{A - B + C * (D * E - F)}{G + H * K}$$

- b. Write a program to evaluate the above arithmetic statement using a stack organized computer with zero-address instructions (so only pop and push can

access memory).

Ans.

a. $A \ B - \ C \ D \ E * \ F - * + \ G \ H \ K * + /$

b. The program would be:

```
Push A
Push B
Subtract
Push C
Push D
Push E
Mult
Push F
Subtract
Mult
Add
Push G
Push H
Push K
Mult
Add
Div
Pop X
```

7- Write code to implement the expression: **$A = (B + C) * (D + E)$**

on 3-, 2-, 1- and 0-address machines. In accordance with programming language practice, computing the expression should not change the values of its operands.

Ans.

Add R1, B, C
Add R2, D, E
Mult A, R1, R2

3-address machine

Load R1, B
Add R1, C
Load R2, E
Add R2, E
Mult R2, R1
Store A, R2

2-address machine

Load B
Add C
Store Temp
Load D
Add E
Mult Temp
Store A

1-address machine

Push B
Push C
Add
Push D
Push E
Add
Mult
Store A

0-address machine